Decision tree vs Decision tables

Datele de identificare ale membrilor echipei

- Toda-Puiuleț Emanuela Coralia 237
- Trifan Alexandra-Isabela 237
- Vicol Marius 237

Enunțul problemei rezolvate

Se dorește testarea modulului de validare a împrumuturilor dintr-o **bibliotecă**. Sistemul trebuie să calculeze automat numărul de zile pentru care un utilizator poate împrumuta o carte, pe baza a trei condiții:

1. **Penalizări:** Utilizatorul are penalizări financiare neplătite din trecut? (Da/Nu)
2. **Tip Carte:** Cartea dorită este o "noutate" (lansată în ultimele 3 luni)? (Da/Nu)
3. **Tip Abonament:** Utilizatorul este elev/student? (Da/Nu)

Reguli de business impuse de bibliotecă:

- R1: Dacă utilizatorul are penalizări neplătite, împrumutul este **Respins (0 zile)**, indiferent de alte condiții.
- R2: Dacă nu are penalizări și cartea este o noutate, elevii/studenții o pot lua pentru **7 zile**, iar abonamentul Standard pentru **14 zile** (pentru a asigura rulajul rapid al cărților noi).
- R3: Dacă nu are penalizări și cartea este veche (nu e noutate), elevii/studenții o pot lua pentru **30 de zile** (pentru referate/sesiune), iar abonamentul Standard pentru **21 de zile**.

Descrierea teoretică a metodelor

A. Decision Tables

Tabelele de decizie reprezintă o tehnică de testare *black-box* și modelare a cerințelor care surprinde reguli de business complexe într-o formă tabelară, extrem de riguroasă. Această tehnică este esențială atunci când rezultatul depinde de o combinație specifică a mai multor condiții de intrare.

Componentele structurale ale unui tabel de decizie:

1. **Condition stub (Cadranul condițiilor):** se listează pe rânduri toți parametrii de intrare (ex. *Are penalizări*, *Este noutate*).

2. **Action stub (Cadranul acțiunilor):** se listează pe rânduri toate rezultatele posibile (ex. *0 zile*, *7 zile*).
3. **Condition entries (Intrările condițiilor):** Coloanele care conțin valorile logice (Adevărat/Fals) pentru fiecare condiție.
4. **Action entries (Intrările acțiunilor):** Marcajele care indică ce acțiune se execută pentru o anumită combinație de condiții.

Metodologia de creare a tabelului:

- Se identifică numărul total de condiții booleene (n).
- Se calculează numărul maxim de reguli teoretice (coloane) folosind formula 2^n. Pentru 3 condiții, avem 2^3= 8 reguli posibile.
- Optimizarea (Rule collapse): Tehnica permite identificarea conceptului de *"Don't Care"* (marcat de obicei cu "-"). Dacă o condiție dictează rezultatul indiferent de celelalte variabile (ex. *Are Penalizări = Da*), regulile redundante se pot comasa, reducând numărul de cazuri de testare necesare de la 8 la 5, fără a pierde acoperirea.

B. Decision Trees

Arborii de decizie reprezintă o tehnică de modelare vizuală, frecvent utilizată atât în ingineria software, cât și în algoritmii de *Machine Learning*. Este un model predictiv și descriptiv care mapează observațiile despre un element la concluzii despre valoarea țintă a acestuia.

Componentele structurale ale unui arbore:

1. **Nodul rădăcină (Root node):** Reprezintă prima și cea mai importantă decizie. Separă setul de date în cele mai mari subsecțiuni.
2. **Nodurile interne (Decision nodes):** Reprezintă testarea unei variabile/condiții intermediare.
3. **Ramurile (Branches):** Conectează nodurile și reprezintă ieșirile testului (ex. ramura "Da" sau ramura "Nu").
4. **Nodurile frunză (Leaf/Terminal nodes):** Punctele finale ale arborelui, care nu se mai ramifică. Ele reprezintă rezultatul final (acțiunea).

Metodologia de creare a arborelui:

- Se alege condiția cu cel mai mare impact pentru a fi plasată la rădăcină (în cazul nostru, *Penalizările*, deoarece ele blochează din start procesul).
- Se creează ramificații secvențiale. Arborele este citit de sus în jos (sau de la stânga la dreapta), validând pas cu pas logica aplicației, ceea ce îl face extrem de ușor de citit de către persoanele non-tehnice (stakeholderi).

Soluția propusă: implementare și cod sursă

Pentru a evidenția diferența dintre cele două tehnici, am implementat sistemul bibliotecii folosind două abordări distincte: o abordare bazată pe date (decision tables) și o abordare bazată pe structuri de control (decision trees).

Cod 1: Implementarea tabelei de decizie (*Data-Driven / Lookup table*)

În programare, un tabel de decizie complet se transpune excelent într-un dicționar (mapare exhaustivă). Această metodă garantează că fiecare din cele 8 reguli teoretice (2^3) are un rezultat clar definit.

```
def calculate_loan_days_table(has_penalties, is_new_release, is_student):
    """
    Data-Driven Approach: The dictionary represents the exact Decision
Table
    (All 8 theoretical rules).
    The dictionary key is a tuple representing the combination:
    (has_penalties, is_new_release, is_student)
    """
    decision_table = {
        (True, True, True): 0,  # R1: Has penalties, other conditions don't
matter
        (True, True, False): 0,  # R2: Has penalties
        (True, False, True): 0,  # R3: Has penalties
        (True, False, False): 0,  # R4: Has penalties
        (False, True, True): 7,  # R5: No penalties, New release, Student
        (False, True, False): 14,  # R6: No penalties, New release,
Standard user
        (False, False, True): 30,  # R7: No penalties, Old book, Student
        (False, False, False): 21  # R8: No penalties, Old book, Standard
user
    }

    # Lookup in the table (O(1) time complexity)
    current_condition = (has_penalties, is_new_release, is_student)
    approved_days = decision_table.get(current_condition, -1)

    return approved_days


# --- Test Case Execution ---
print("Table Result (Penalties=False, NewRelease=True, Student=False):",
      calculate_loan_days_table(False, True, False), "days")
```

Reprezentare vizuală:

A. Tabelul de decizie complet (toate cele 8 combinații posibile)

Condiții / Reguli	R1	R2	R3	R4	R5	R6	R7	R8

Are Penalizări?	Da	Da	Da	Da	Nu	Nu	Nu	Nu
Carte Noutate?	Da	Da	Nu	Nu	Da	Da	Nu	Nu
Este Student/Elev?	Da	Nu	Da	Nu	Da	Nu	Da	Nu

ACȚIUNI (Rezultat)

0 Zile (Respins)	X	X	X	X				
7 Zile					X			
14 Zile						X		
21 Zile								X
30 Zile							X	

B. Tabelul de decizie optimizat (tehnica "Don't Care")

Condiții / Reguli de Testare	Test 1 (Blocaj)	Test 2	Test 3	Test 4	Test 5
Are Penalizări?	Da	Nu	Nu	Nu	Nu
Carte Noutate?	-	Da	Da	Nu	Nu
Este Student/Elev?	-	Da	Nu	Da	Nu

Rezultat Așteptat	0 Zile	7 Zile	14 Zile	30 Zile	21 Zile

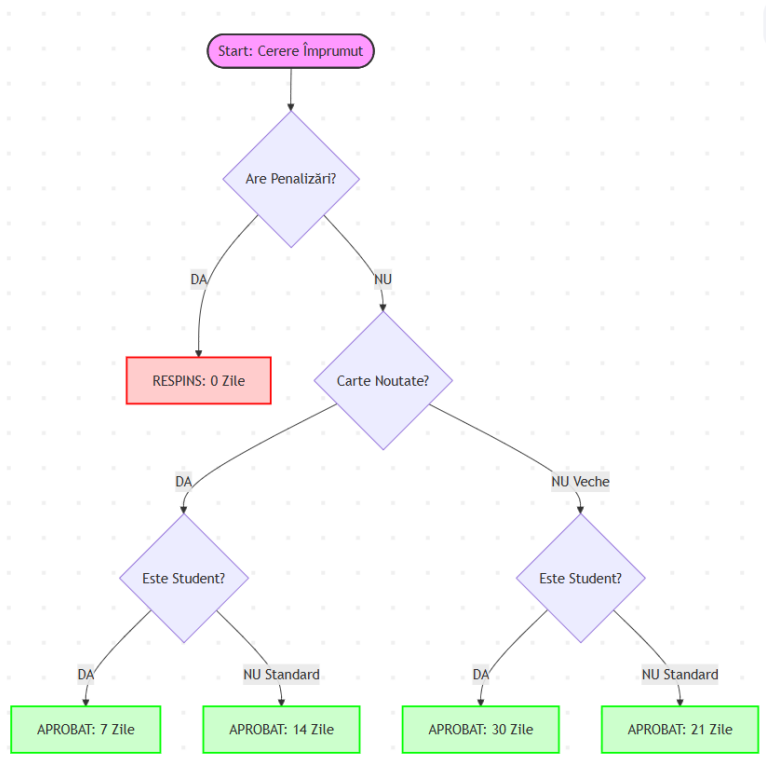
Cod 2: Implementarea arborelui de decizie (*Control-Flow / Nested Logic*)

Această implementare reflectă exact ramificarea grafică a unui arbore de decizie. Se observă cum deciziile sunt luate secvențial, iar execuția se oprește la prima "frunză" întâlnită.

```
def calculate_loan_days_tree(has_penalties, is_new_release, is_student):
    """
    Control-Flow Approach: The code structure visually represents the
branches
```

```
    of the decision tree. It leverages the "Fast-fail" / "Don't care"
concept
    intrinsic to the tree logic.
    """
    # ROOT NODE: The most critical condition
    if has_penalties == True:
        return 0  # LEAF NODE: The path ends here (Loan rejected)
    else:
        # INTERNAL NODE 1: Check the type of the book
        if is_new_release == True:

            # INTERNAL NODE 2 (Left Branch): Check user subscription
            if is_student == True:
                return 7  # LEAF NODE
            else:
                return 14  # LEAF NODE

        else:  # The book is an older release

            # INTERNAL NODE 2 (Right Branch): Check user subscription
            if is_student == True:
                return 30  # LEAF NODE
            else:
                return 21  # LEAF NODE


# --- Test Case Execution ---
print("Tree Result (Penalties=False, NewRelease=True, Student=False):",
      calculate_loan_days_tree(False, True, False), "days")
```

Reprezentare vizuală:



Analiză Comparativă și Recomandări

1. Analiză Decision Tables

Puncte Forte:

- **Exhaustivitate și acoperire (Coverage):** Prin generarea tuturor combinațiilor posibile (2^n), tabelul garantează o acoperire de 100% a cerințelor. Este instrumentul ideal pentru a descoperi defecte de tip „condiție omisă" (missing requirements) de către analiștii de business.
- **Optimizare prin tehnica „Don't Care":** Capacitatea de a comasa regulile redundante reduce masiv efortul de testare (în exemplul nostru, de la 8 la 5 teste) fără a compromite calitatea validării.
- **Trasabilitate directă în cod:** Tabelele se pot transpune direct în structuri de date de tip dicționar sau matrice (*Data-Driven Testing*), făcând codul extrem de curat și ușor de întreținut.

Dezavantaje:

- **Explozia combinatorie:** Scalabilitatea este o problemă majoră. Dacă sistemul ar avea 10 condiții, tabelul ar genera 1024 de coloane (2^{10}), devenind imposibil de citit și gestionat manual.
- **Lipsa perspectivei secvențiale:** Tabelul nu oferă nicio informație despre ordinea cronologică în care ar trebui evaluate condițiile în algoritmul aplicației.

2. Analiza Decision Trees

Puncte Forte:

- **Intuitivitate și comunicare vizuală:** Datorită naturii lor grafice, arborii sunt excelenți pentru a explica logica de funcționare stakeholderilor non-tehnici (clienți, manageri).
- **Eficiență în execuție (Fast-Fail):** Reprezentarea obligă programatorul să gândească secvențial. Condițiile critice (precum *Penalizările*) sunt plasate la rădăcină, oprind fluxul inutil de verificări (short-circuiting), optimizând astfel performanța sistemului.
- **Modelare naturală a deciziilor asimetrice:** Sunt perfecți atunci când o anumită ramură necesită mult mai multe verificări decât alta.

Dezavantaje:

- **Dificultate în mentenanță:** O mică modificare a unei condiții de la baza arborelui (rădăcină) poate necesita redesenarea completă a întregii structuri.

- **Riscul de a omite ramuri:** Vizual, dacă arborele devine foarte „stufos”, este ușor ca echipa de testare să omită o cale (o frunză) care nu a fost explicit desenată.

Obiective de testare recomandate:

Se recomandă utilizarea **Decision Tables** pentru modulele de *backend*, calcule financiare sau sisteme critice unde siguranța matematică și lipsa ambiguității sunt vitale. **Decision Trees** sunt recomandați pentru modelarea fluxurilor de utilizator (*User Journeys* în *frontend*), sisteme de suport clienți sau diagnoză, unde ordinea pașilor este mai importantă decât evaluarea simultană a tuturor variabilelor.

Criteriu de Analiză	Decision Tables	Decision Trees
Puncte Forte	- Nu permite omiterea niciunei combinații posibile (exhaustivitate). - Facilitează descoperirea conceptului de "Don't care"	- Ușor de înțeles de către personalul non-IT. - Oferă o ordine cronologică clară a validărilor.
Dezavantaje	- Greu de citit dacă numărul de reguli depășește 15-20. - Nu arată ordinea în care trebuie executate verificările în cod.	- Complexitatea vizuală crește exponențial (aspect stufos). - Greu de observat dacă o frunză (un rezultat) a fost omisă accidental.

Resurse Folosite

Medii de dezvoltare și instrumente software:

1. JetBrains PyCharm
2. mermaid.live: instrument de modelare vizuală utilizat pentru proiectarea și exportul diagramei specifice arborelui de decizie.

Resurse Web:

https://www.geeksforgeeks.org/software-engineering/decision-table-based-testing-in-software-testing/

https://www.geeksforgeeks.org/software-engineering/decision-tree-in-software-engineering/